

Manipulação de Registros

A figura a seguir ilustra registros de um arquivo de dados com um indicador de tamanho de uma entidade de banco de dados (150 bytes, onde 'b' representa espaço em branco). Assim, a tabela possui os seguintes campos: nome, endereço, número e bairro. Por fim, o número -1 sinaliza o fim dos registros. Realize o que se pede:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29		
5					M	A	R	I	A	5					R	U	A	b	1	4				123				10			
30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59		
S	A	O	b	C	A	R	L	O	S	4					J	O	A	O	5				R	U	A	b	A	4			
60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89		
			255			9				R	I	O	b	C	L	A	R	O	5				P	E	D	R	O	6			
90	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119		
		R	U	A	b	X	V	4				56					10				S	A	O	b	C	A	R	L	O	S	
120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149		
-1																															

a) Implemente uma classe para a entidade com todos os campos encapsulados como atributos: nome (*String*), endereço (*String*), número da casa (*int*) e bairro (*String*), utilizando métodos getters e setters .

b) Crie um método na referida classe, utilizando a classe *RandomAccessFile*, que receba como parâmetro uma *string* representando o caminho do arquivo e o RRN (*Relative Record Number*). O método deve atribuir os valores aos atributos da classe, no arquivo. Caso o arquivo não seja encontrado ou o registro não seja localizado, uma exceção deverá ser lançada.

c) Crie um método estático na classe que receba como parâmetro uma *string* representando o caminho do arquivo que seja capaz de listar todos os registros da entidade de forma organizada (*RandomAccessFile*).

d) Crie um método na referida classe, utilizando a classe *RandomAccessFile*, que receba como parâmetro uma *string* representando o caminho do arquivo. O procedimento deve localizar o byte do último registro e salvar os atributos da classe no arquivo. Não se esqueça de atribuir o valor -1 ao final do arquivo para indicar o fim dos registros.

Programa Java: lê e armazena segundo valor

suas respectivas explicações:

Tabela de Comandos do `RandomAccessFile`

Método	Descrição	Exemplo
Construtores		
<code>RandomAccessFile(String nome, String modo)</code>	Abre um arquivo no modo especificado ("r", "rw", "rws", "rwd").	<code>RandomAccessFile arq = new RandomAccessFile("dados.dat", "rw");</code>
Posicionamento do Cursor		
<code>long getFilePointer()</code>	Retorna a posição atual do cursor (em bytes).	<code>long pos = arq.getFilePointer();</code>
<code>void seek(long pos)</code>	Movimenta o cursor para uma posição específica (em bytes).	<code>arq.seek(100); // Salta para o byte 100</code>
Leitura de Dados		
<code>int read()</code>	Lê um único byte (-1 se fim do arquivo).	<code>int byte = arq.read();</code>
<code>int read(byte[] b)</code>	Lê bytes e armazena no array b.	<code>byte[] buffer = new byte[10]; arq.read(buffer);</code>
<code>readInt()</code>	Lê um int (4 bytes).	<code>int num = arq.readInt();</code>
<code>readFloat()</code>	Lê um float (4 bytes).	<code>float valor = arq.readFloat();</code>
<code>readDouble()</code>	Lê um double (8 bytes).	<code>double d = arq.readDouble();</code>
<code>readChar()</code>	Lê um char (2 bytes).	<code>char c = arq.readChar();</code>
<code>readBoolean()</code>	Lê um boolean (1 byte).	<code>boolean b = arq.readBoolean();</code>
<code>readUTF()</code>	Lê uma String no formato UTF.	<code>String texto = arq.readUTF();</code>

Para descontrair

